(Gaussian) Mixture Models

Motivation

- k-Means has Flaws, like ...
  - being **sensitive** to the initialization
  - only working well when...
    - *circular data* (based on the distance - only the center matters)
    - clusters having *roughly the same size*
    - clusters are well seperated - “hard” assignment without double assignments
- Similarly to Classification with Logistic Regression, we can introduce the **notion of uncertainty to k-means**
  → “*Overlapping Clusters*”, where in overlap regions there is some uncertainty to what cluster a data point belongs too!
  - We want to have “soft” assignments of observations to clusters

How can we attack these painpoints?

- We can use **mixture models** to account for the drawbacks of regular k-means clustering
  - We can model **overlapping distributions** of specific features, where clustering & grouping can be hard (e.g. the color of images with different themes)
  - We can have clusters of **different shapes** & **different weights** based on how often they occur

→ Allows for a model that is a bit more flexible and not as restricted!

Definition

A GMM is a **generative model** that describes a dataset's probability distribution, $P(x)$, as a mixture of several weighted Gaussian "blobs."

Each **cluster** is modeled by *relative frequency of its occurence, mean and standard variance* → π_k (relative freq.), μ_k (mean), σ_k (std)

→ This provides a **"soft" clustering approach** that is more flexible than k-means, able to model clusters of varying shapes (ellipses) and sizes.

Generative Models

Generative models are able to generate new datapoints as well, as they learn the joint probability distribution of different variables.

1. First, a cluster k is chosen based on the mixture weights (prior probabilities, π_k)
2. Then the datapoint x is drawn from the specific cluster’s Gaussian distribution, defined by mean μ_k and covariance Σ_k.

   The challenge here is, that in the unsupervised setting, we are missing the true cluster assignments for each point. This makes this problem harder to solve and requires the EM-Algorithm

Learning Algorithm

- The **hidden variable** z encodes for each datapoint x, what specific Gaussian cluster it belongs to
  - But as when we learn from unknown data, z is usually hidden to us!

- **Responsibility** r_{ik} → *Probability* of assignment to a cluster k given **model params** θ **& observed value** x
  - Includes initial probability/bias to belong to cluster k, being pi_k - and the likelihood of that observation in the cluster k
  **→ Would represent the "soft assignments" for data points; our estimateion of the ground truth ("hard assignments")

How do we calculate now cluster assignments AND cluster parameters?

- It's easy to calculate cluster parameters, knowing in before hand...
  - **the hard (cluster) assignments** → We can then, as in SVL, just MLE for the best fitting parameters! → But we don't have access to them!
  - **The responsibilities!** → Same here, we can, per cluster, take the responsibility for each datapoint and use it as a weight when calculating the **cluster parameters (mean, variance & relative freq \pi)!**
- How can we not need to calculate both at the same time? We could go for an **iterative algorithm, similar to k-means!**
  - We update each element one after another, and slowly converging to a likelihood maximum

Expectation Maximization (EM) for GMMs

Iterative algorithm, with two steps per iteration

1. **E-Step** → **E**stimate the cluster responsibilities given current parameter estiamtes
2. **M-step → M**aximize likelihood over parameters given **current responsibilities** to find a new \Theta

→ K-means can be seen as a special case of EM for GMMs where all clusters are assumed to be simple spheres and assignments are all-or-nothing (and where we use hard assignments)!

k-Nearest Neighbors

→ Find k closest data points based on the e.g. distance

- We average the neighbors labels to predict the output for new **datapoints**
- **Non-parametric model** → We calculate for every input the closest neighbors, to calculate the output

Decision Trees

→ Hypothesis that consists of multiple if/else rules; ✅ Really interpretable and understandable, what happens!; And very simple - splits along feature axis! ❌ Generalize poorly as they usually overfit (high bias but too simple!), and highly unstable around boundaries

Types of Trees

- **Classification Trees →** Give us in the *leaf nodes* a **specific discrete label**
- **Regression Trees →** Give us a **continuous number!**
- Trees can split on **binary** or **real-values** input features
  - Binary features are relatively straightforward split into True/False
  - Real-valued features do have a **finite number of points that we should consider**!
    - These are the mid-points in between the (sorted) feature values of different datapoints → n & (n-1) possibilities
    - We can just take the best mid-point!

> 💡 **How can we encode uncertainty with a specific classification/regression?**
>
> For classification, this is relatively straightforward, based on the misclassification at that specific node, we can use the odds between two classes as a uncertainty measure

How do we learn a tree?

- ! No closed form approach to come up with the best tree
    - Computationally intractable to enumerate all different types of trees & find the best performing tree

Cost Function Structure

- **Measure of Fit:** Different ways to measure performance of a decision tree → **Fraction of mistakes**, **Entropy**
- **Measure of Complexity:** Number of specific leaf nodes, as the more they are, the easier we can predict the true labels for each datapoint

Intuition

- Starting out with one root-node & doing the uninformed guess on taking the class of the bigger fraction, we get a **baseline of classification error**!
  - Then we can try out different features and see how well they are splitting at the stumps, still based on a majority vote
- *When do we know when we don't want to continue splitting?*
  1. We don't have any features to split on anymore
     - 💡 With classification, the *amount of splits* is limited by whether our features are continuous or discrete!
  2. We have a perfect split already (all datapoints in the node have the same y-value)!
     - Or because we want to avoid *overfitting* we might want to stop earlier!
       - ⚠️ We don't want too many leaves, as if the amount of leaves equals the datapoints we have you can of course classify each perfectly - but that doesn't generalize well!
         - We can alternatively also disregard this, and prune after having learned the full tree, and simplify this to a simpler representation; could be better as we might not miss out on any "generalized" classifications

⑦ How can we avoid overfitting and get simpler trees?

- Stopping the splitting algorithm before the tree becomes too complex
- Simplify the tree after at some point we classified everything perfectly

Ensemble Models

→ General overarching philosophy, not only limited to decision trees. **The core idea** is that by combining many *simple, imperfect models* (called "weak learners"), we can create a *single, powerful "strong learner*

- **Motivation:** If a single tree tends to *overfit on specific data*, what happens if I train a lot of different trees on data, and take the majority vote of their predictions?
  → We can take advantage of the low bias (high-variance) setting to *estimate* the signal much better!
- **Drawbacks:** Reduced interpretability, increased complexity & initial training

💡 Trees as a higher-level linear regression

- We can see each *prediction* of a tree as a **specific feature,** and expand that problem basically to its own, higher-level regression problem (as the trees individually perhaps have different performances and importance in the final signal)

Goal	Learn ensemble model	Prediction
• Predict output y • Either $+1$ or -1 • From input $\mathbf{x}$	• Classifiers: $f_1(\mathbf{x}), f_2(\mathbf{x}), \ldots, f_T(\mathbf{x})$ • Coefficients: $\hat{w}_1, \hat{w}_2, \ldots, \hat{w}_T$	$\hat{y} = \text{sign}\left(\sum_{t=1}^{T} \hat{w}_t f_t(\mathbf{x})\right)$

Bagging vs. Boosting

> **This applies to all sorts of models that *learn* - not only trees!**

Bagging (Bootstrap Aggregating) - Reducing Variance

→ One specific implementation of Ensemble Learning

1. We **bootstrap** a lot of different datasets by randomly sampling from our initial dataset
2. Then we are training a lot of shallow, deep trees
   - Each tree has now a slightly different "lens" towards some features/data, and effectively learning a bit of different patterns & overfit in a different way! (important to make use of the low bias)
3. By taking the average prediction of all these different trees, we can average out their errors & ***reduce the variance*** of the trees to make them perform better on unseen data!

Boosting - Reducing Bias

- Combining many simple, *high-bias/low-variance models* (shallow trees) sequentially, that each specialize in a specific thing, to reduce the **inherent bias** of these trees
- We can make this work by training one model, somehow model on what examples/features it performs bad, and to train another model that tries to go on about that
  - And when we know what it performs bad on? We can just give them a specific subset of examples it performs bad on

> **When to use what?**
>
> - Bagging makes a lot of sense when we have a noisy & small dataset, where we want to reduce variance with very complex models
> - Boosting makes mostly sense to relatively clean datasets, and when our models are high-bias and cannot fully contort to the data's patterns; Noisy datasets can lead to problems as the high-bias learners might focus on examples & features that itself are very noisy

Algorithmic Implementations

Adaboost

- We are training iteratively one by one a new classifier
  - We train a classifier on a dataset (and its weights, which are at the beginning uniform)
  - We can **calculate** a **classifier's weight**, based on its classification error on all data points
    - If it has a *low error*, it has a high positive weight
    - With a *high error*, it has a high negative weight
    - If it has an error around 0.5, it doesn't give us any proper information, so the weight is close to zero
  - The datapoint's weight, **alpha,** that changes per iteration of a classifier puts a larger weight on datapoints that we get wrong, and smaller weight on things we get right

- We do this based on the weight of the current classifier we trained, as we want to include how well it is classifying the examples it gets right!
- We also *normalize* the weights as they can really large & small after some iteration!

Random Forests

→ Building upon Bagging

